Undergraduate Senior Thesis

University of Illinois Urbana-Champaign

The Grainger College of Engineering Department of Computer Science

Algorithmic Methods for Ordering Reconstruction and Path Optimization in Directed Acyclic Graphs

An Undergraduate Thesis on Topological Sorting and Dynamic Programming

Submitted by

Joseph Jaeyun Jeong

Thesis Advisor: Professor Nickvash Kani

Undergraduate Senior Thesis

May 2024

Abstract

Directed graphs provide a flexible representation of precedence constraints and relationships among objects. A directed acyclic graph (DAG) is especially useful for algorithm design because its vertices admit a topological ordering, allowing many graph problems to be solved without the complications introduced by directed cycles. This thesis studies four related algorithmic tasks: reconstructing an unknown symbol order from a lexicographically sorted dictionary, computing shortest and longest paths in a weighted DAG, and finding minimum-weight paths subject to upper-bound or exact constraints on the number of edges.

The study presents graph formulations, algorithmic procedures, correctness arguments, and asymptotic complexity analyses. For alphabet reconstruction, adjacent dictionary entries are used to infer precedence constraints, after which topological sorting produces an ordering consistent with those constraints. For path optimization in a DAG, dynamic relaxation in topological order yields shortest and longest path values in linear time, even when edge weights are negative. For edge-count-constrained paths, a dynamic programming recurrence tracks the number of edges used and distinguishes paths with at most k edges from paths with exactly k edges.

The thesis also clarifies assumptions required for each result and identifies important implementation details, including detection of inconsistent ordering constraints, treatment of unreachable vertices, and the distinction between exact-edge and at-most-edge dynamic programming states. The evaluation in this thesis is theoretical: no experimental measurements are claimed. The results illustrate how graph structure and dynamic programming provide systematic and efficient solutions to ordering and path optimization problems.

Keywords: directed acyclic graph, topological sorting, alphabet reconstruction, shortest path, longest path, dynamic programming, algorithm analysis.

Contents

Contents ii

List of Tables iv

1 Introduction 1
- 1.1 Background and Motivation . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Research Questions and Objectives . . . 2
- 1.4 Scope and Contributions . . . 2
- 1.5 Thesis Organization . . . 3

2 Background and Algorithmic Preliminaries 4
- 2.1 Directed Graphs and Weighted Paths . . . 4
- 2.2 Topological Ordering . . . 4
- 2.3 Algorithm Analysis . . . 5
- 2.4 Dynamic Programming Principle . . . 5

3 Reconstructing an Unknown Symbol Order 6
- 3.1 Problem Definition . . . 6
- 3.2 Graph Construction . . . 6
- 3.3 Algorithm . . . 7
- 3.4 Correctness Analysis . . . 7
- 3.5 Complexity Analysis . . . 8

4 Shortest and Longest Paths in a DAG 9
- 4.1 Problem Definition . . . 9
- 4.2 Shortest Paths . . . 9
  - 4.2.1 Algorithm Design . . . 9
  - 4.2.2 Correctness Proof . . . 10

4.2.3 Complexity Analysis . . . 10
4.3 Longest Paths . . . 11
4.3.1 Algorithm Design . . . 11
4.3.2 Correctness Proof . . . 11
4.3.3 Complexity Analysis . . . 12
4.4 Discussion of Negative Weights . . . 12

5 Shortest Paths with Edge-Count Constraints 13
5.1 Problem Setting . . . 13
5.2 Paths with at Most k Edges . . . 13
5.2.1 State Definition and Recurrence . . . 13
5.2.2 Algorithm . . . 14
5.2.3 Correctness Proof . . . 14
5.2.4 Complexity Analysis . . . 15
5.3 Paths with Exactly k Edges . . . 15
5.3.1 State Definition and Recurrence . . . 15
5.3.2 Algorithm . . . 15
5.3.3 Correctness Proof . . . 16
5.3.4 Complexity Analysis . . . 16
5.4 Comparison of the Two Dynamic Programs . . . 16

6 Comparative Discussion and Limitations 18
6.1 Summary of Theoretical Complexity . . . 18
6.2 Assumptions and Implementation Considerations . . . 18
6.3 Theoretical Evaluation and Empirical Limitations . . . 19
6.4 Limitations . . . 19

7 Conclusion and Future Work 20
7.1 Conclusion . . . 20
7.2 Future Work . . . 20

References 22

A Suggested Verification Checklist 23

List of Tables

5.1 Comparison of edge-count-constrained shortest-path formulations 16

6.1 Summary of the algorithms analyzed in this thesis 18

Chapter 1

Introduction

1.1 Background and Motivation

Graphs are widely used to represent relationships, dependencies, and precedence constraints. In a directed graph, an edge from one vertex to another expresses an asymmetric relationship. Directed acyclic graphs are a particularly important class because they represent dependencies without circular precedence requirements. Examples include task scheduling, build systems, dependency resolution, and staged computations.

A topological ordering of a DAG places every vertex before all vertices reachable from it by a single outgoing edge. This ordering gives a natural processing sequence: when a vertex is processed, all of its immediate predecessors have already been processed. That property supports efficient algorithms for a variety of problems, including path optimization.

This thesis examines several algorithmic problems that share this structural foundation. The first reconstructs an unknown ordering of symbols from a dictionary already sorted according to that order. The remaining problems optimize path weights in directed graphs, with particular attention to DAGs and to constraints on the number of edges in a path.

1.2 Problem Statement

The thesis considers four tasks:

1. Infer precedence constraints among symbols from adjacent words in a lexicographically sorted dictionary and recover a consistent symbol ordering.

2. Compute minimum- and maximum-weight paths from a source vertex in a weighted DAG, allowing positive, zero, and negative edge weights.

3. Find a minimum-weight path between two vertices that uses at most k edges.

4. Find a minimum-weight path between two vertices that uses exactly k edges.

The first task is an ordering problem, while the other three are path optimization problems. The two edge-count-constrained problems are similar, but their state definitions and transitions differ in a way that is essential to correctness.

1.3 Research Questions and Objectives

The following questions guide the analysis:

- How can lexical comparisons be converted into graph constraints, and under what conditions does topological sorting recover a valid symbol order?
- Why does topological processing permit shortest- and longest-path algorithms to handle negative edge weights in a DAG?
- How should dynamic programming states be defined to distinguish paths using at most k edges from paths using exactly k edges?
- What are the time and auxiliary-space requirements of the resulting algorithms?

The objectives are to formalize each problem, present an implementable algorithm, justify its correctness, and derive its asymptotic complexity.

1.4 Scope and Contributions

This thesis provides a unified theoretical treatment of the four tasks. Its contributions are:

1. a graph-based formulation of alphabet-order reconstruction, including the conditions under which the result is valid or unique;
2. topological-order algorithms for shortest and longest paths in weighted DAGs, together with inductive correctness proofs;
3. separate dynamic programming formulations for at-most-k and exactly-k-edge path optimization; and
4. a comparative complexity analysis and a discussion of implementation assumptions and limitations.

The work does not claim a new algorithmic result. It organizes standard algorithmic techniques and makes their assumptions and distinctions explicit. No empirical performance claims are made because no experimental data are provided.

1.5 Thesis Organization

Chapter 2 introduces the graph concepts and algorithmic principles used throughout the thesis. Chapter 3 presents alphabet reconstruction. Chapter 4 develops shortest- and longest-path algorithms for DAGs. Chapter 5 studies path optimization under edge-count constraints. Chapter 6 compares the methods and discusses limitations. Chapter 7 concludes the thesis and identifies possible future work.

Chapter 2

Background and Algorithmic Preliminaries

2.1 Directed Graphs and Weighted Paths

Definition 2.1. A directed graph is a pair $G = (V, E)$, where V is a finite set of vertices and $E \subseteq V \times V$ is a set of directed edges. A weighted directed graph additionally has a weight function $w : E \to \mathbb{R}$.

A directed path from s to t is a sequence of vertices beginning at s and ending at t, with each consecutive pair connected by a directed edge. The weight of a path is the sum of the weights of its edges. A path with no edges has weight zero and begins and ends at the same vertex.

Definition 2.2. A directed cycle is a nonempty directed path whose first and last vertices are the same. A directed acyclic graph (DAG) is a directed graph containing no directed cycle.

2.2 Topological Ordering

Definition 2.3. A topological ordering of $G = (V, E)$ is a linear ordering $v_1, \ldots, v_{|V|}$ of its vertices such that for every edge $(v_i, v_j) \in E$, $i < j$.

A directed graph has a topological ordering if and only if it is acyclic. Algorithms such as Kahn's algorithm and depth-first-search-based ordering compute a topological order in $O(|V| + |E|)$ time. If the graph contains a directed cycle, a complete topological ordering does not exist.

A topological ordering is not necessarily unique. If an application requires a uniquely determined order, it must establish that the constraints admit exactly one topological ordering; an arbitrary topological sort alone does not establish uniqueness.

2.3 Algorithm Analysis

Let $|V|$ and $|E|$ denote the number of vertices and edges. The analysis uses asymptotic upper bounds in terms of these quantities and, where relevant, the number of words n, word length m, alphabet size $|\Sigma|$, and edge limit k. Unless otherwise stated, the graph is represented using adjacency lists, and arithmetic operations on edge weights are treated as constant-time operations.

2.4 Dynamic Programming Principle

Dynamic programming solves a problem by defining subproblems whose solutions can be combined to solve larger instances. A rigorous formulation specifies the state, base case, recurrence, order of evaluation, and final answer. In the edge-count-constrained problems, the number of edges used is part of the state. This prevents an at-most-k solution from being confused with an exactly-k solution.

Chapter 3

Reconstructing an Unknown Symbol Order

3.1 Problem Definition

Let $D = (d_1, d_2, \ldots, d_n)$ be a dictionary sorted lexicographically under an unknown alphabet Σ. The task is to infer an ordering of the symbols consistent with the dictionary. The input may not uniquely determine every pairwise relation, so the goal in the general case is to return a valid ordering consistent with the constraints that can be inferred.

For clarity, the algorithm below works for words of possibly different lengths. If two words share a common prefix and the first word is longer than the second, the dictionary violates ordinary lexicographic order; this invalid-prefix case must be detected. The original fixed-length-word setting does not encounter that case.

3.2 Graph Construction

Construct a directed graph $G = (V, E)$ with one vertex for each symbol in Σ. For each adjacent pair (d_i, d_{i+1}), find the first position at which the words differ. If that position contains symbols a and b, respectively, add the edge (a, b). This edge represents the constraint that a precedes b.

Only the first differing position is relevant: the lexical comparison is determined at that position, and later characters cannot reverse the comparison. Examining adjacent pairs is sufficient because a sorted list's adjacent comparisons encode the ordering constraints needed to validate the entire sequence.

3.3 Algorithm

```
Algorithm RecoverAlphabet(D, Sigma):
    G = graph with vertex set Sigma and no edges
    for i = 1 to length(D) - 1:
        a = D[i]
        b = D[i + 1]
        j = first position where a and b differ
        if such a position exists:
            add edge (a[j], b[j]) to G
        else if length(a) > length(b):
            report invalid lexicographic input
    if G contains a directed cycle:
        report inconsistent constraints
    return a topological ordering of G
```

Listing 3.1: Reconstructing a symbol order from a sorted dictionary

Duplicate edges may be ignored. If the graph is acyclic, any topological ordering satisfies the inferred precedence constraints. If uniqueness is required, the implementation must additionally check whether the topological ordering is unique.

3.4 Correctness Analysis

Lemma 3.1. *For any adjacent pair of words that differ at a first position j, the edge added by the algorithm is a necessary precedence constraint for any alphabet ordering consistent with the dictionary.*

Proof. The two words agree at every position before j. Their lexical order is therefore determined by the symbols at position j. Since d_i precedes d_{i+1}, the symbol $d_i[j]$ must precede $d_{i+1}[j]$. The edge added by the algorithm records precisely this relation. □

Theorem 3.2. *If the input dictionary is valid and the graph constructed by the algorithm is acyclic, every topological ordering returned by the algorithm is consistent with all precedence constraints inferred from adjacent dictionary entries.*

Proof. By the lemma, every edge represents a precedence constraint required by the lexical comparison of an adjacent pair. A topological ordering places the tail of every edge before its head. Thus, the returned order satisfies every constraint recorded in the graph. Since each adjacent pair is examined, all constraints obtained from the dictionary by this comparison

procedure are represented. The returned order is therefore consistent with all inferred constraints. □

Remark 3.3. The constraints may admit more than one topological ordering. Consequently, the algorithm guarantees a consistent order, not necessarily a uniquely recoverable original alphabet. A cycle indicates inconsistent constraints; it does not produce a valid alphabet order.

3.5 Complexity Analysis

Let m be the maximum word length. Comparing each adjacent pair takes $O(m)$ time, so graph construction takes $O(nm)$ time. Topological sorting takes $O(|\Sigma| + |E|)$. At most $n - 1$ comparison edges are generated before duplicate removal. The total time is therefore

$$O\Big(nm + |\Sigma| + |E|\Big) = O\Big(nm + |\Sigma| + n\Big),$$

and the graph representation requires $O(|\Sigma| + |E|)$ space. The time to store the input dictionary is excluded from this auxiliary-space bound.

Chapter 4

Shortest and Longest Paths in a DAG

4.1 Problem Definition

Let $G = (V, E)$ be a weighted DAG with edge-weight function $w : E \to \mathbb{R}$ and source vertex s. For each vertex v, the shortest-path problem asks for the minimum weight among all paths from s to v. The longest-path problem asks for the maximum weight among those paths. If v is unreachable from s, the corresponding value is undefined and is represented algorithmically by $+\infty$ for shortest paths and $-\infty$ for longest paths.

Unlike shortest paths in general graphs, DAG path optimization does not require nonnegative edge weights. A DAG has no directed cycles, so there are no cycle-based path repetitions that can make a path weight unbounded. Topological order ensures that every predecessor of a vertex is processed before that vertex.

4.2 Shortest Paths

4.2.1 Algorithm Design

Let π be a topological ordering of G. Initialize $dist[s] = 0$ and $dist[v] = +\infty$ for every $v \neq s$. Process vertices in topological order. For each outgoing edge (u, v) from a reachable vertex u, perform the relaxation

$$dist[v] \leftarrow \min\{dist[v],\ dist[u] + w(u, v)\}.$$

A predecessor pointer may also be stored whenever a value improves, allowing the actual shortest path to be reconstructed.

```
Algorithm DAGShortestPaths(G, s):
    pi = TopologicalSort(G)
    for each vertex v in V:
        dist[v] = +infinity
        parent[v] = NIL
    dist[s] = 0
    for each vertex u in pi:
        if dist[u] != +infinity:
            for each edge (u, v) in OutgoingEdges(u):
                candidate = dist[u] + w(u, v)
                if candidate < dist[v]:
                    dist[v] = candidate
                    parent[v] = u
    return dist, parent
```

Listing 4.1: Shortest paths in a weighted DAG

4.2.2 Correctness Proof

Theorem 4.1. *Algorithm 4.1 computes the shortest-path distance from s to every vertex reachable from s in a weighted DAG.*

Proof. We use induction over the topological ordering. At the source s, the empty path has weight zero, so $dist[s] = 0$ is correct. Vertices not reachable from s retain $+\infty$.

Consider a vertex $v \neq s$ when it is processed. Every incoming edge (u, v) has its tail u earlier in the topological order. By the inductive hypothesis, when u was processed, $dist[u]$ was the shortest-path distance to u, if u was reachable. Relaxing (u, v) therefore considers the weight of a shortest path to u followed by this edge. Every path from s to v ends with some incoming edge (u, v); hence the minimum over all such candidates is exactly the shortest-path distance to v. If no incoming edge can be reached from s, no path to v exists and its distance remains infinite. The claim follows by induction. □

4.2.3 Complexity Analysis

Topological sorting takes $O(|V| + |E|)$ time. Initializing the arrays takes $O(|V|)$ time, and each edge is relaxed at most once. The total running time is $O(|V| + |E|)$. The distance and predecessor arrays require $O(|V|)$ auxiliary space, excluding the graph representation.

4.3 Longest Paths

4.3.1 Algorithm Design

The longest-path problem is difficult in general directed graphs, but it is tractable in DAGs. Initialize $longest[s] = 0$ and $longest[v] = -\infty$ for every other vertex. In topological order, process each outgoing edge using

$$longest[v] \leftarrow \max\{longest[v],\ longest[u] + w(u, v)\}.$$

```
Algorithm DAGLongestPaths(G, s):
    pi = TopologicalSort(G)
    for each vertex v in V:
        longest[v] = -infinity
        parent[v] = NIL
    longest[s] = 0
    for each vertex u in pi:
        if longest[u] != -infinity:
            for each edge (u, v) in OutgoingEdges(u):
                candidate = longest[u] + w(u, v)
                if candidate > longest[v]:
                    longest[v] = candidate
                    parent[v] = u
    return longest, parent
```

Listing 4.2: Longest paths in a weighted DAG

4.3.2 Correctness Proof

Theorem 4.2. *Algorithm 4.2 computes the maximum path weight from s to every vertex reachable from s in a weighted DAG.*

Proof. The empty path establishes the correct value $longest[s] = 0$, and all other vertices begin unreachable. Suppose the vertices before v in topological order have correct longest-path values. Every path from s to v must end with an incoming edge (u, v), where u appears before v. By the inductive hypothesis, $longest[u]$ is the maximum weight of a path to u. The algorithm considers $longest[u] + w(u, v)$ for each reachable predecessor and retains the maximum. This is the maximum weight among all paths to v. If no predecessor is reachable, no path to v exists. Induction proves the claim for all vertices. □

4.3.3 Complexity Analysis

The longest-path algorithm also processes each vertex and edge a constant number of times after topological sorting. Its running time is $O(|V| + |E|)$, and its auxiliary space requirement is $O(|V|)$, excluding the graph representation.

4.4 Discussion of Negative Weights

Both algorithms remain correct when edge weights are negative because their correctness depends on acyclicity and topological order, not on nonnegative weights. The relaxation operation differs only in whether it takes a minimum or maximum. This is a useful distinction from algorithms whose correctness relies on nonnegative edge weights.

Chapter 5

Shortest Paths with Edge-Count Constraints

5.1 Problem Setting

Let $G = (V, E)$ be a weighted directed graph, with source s, destination t, and nonnegative integer k. We consider two variants: a minimum-weight path with at most k edges, and a minimum-weight path with exactly k edges. The dynamic programming algorithms in this chapter work even when the graph is not acyclic, because the edge-count bound makes the number of transitions finite. Edge weights may be negative.

5.2 Paths with at Most k Edges

5.2.1 State Definition and Recurrence

Define $A_q[x]$ as the minimum weight of any path from s to x using at most q edges. If no such path exists, set $A_q[x] = +\infty$. The base case is

$$A_0[x] = \begin{cases} 0, & x = s, \\ +\infty, & x \neq s. \end{cases}$$

For $q \geq 1$, a path using at most q edges either uses at most $q-1$ edges, or extends a path using at most $q-1$ edges by one edge. Thus,

$$A_q[v] = \min \left\{ A_{q-1}[v],\ \min_{(u,v) \in E} (A_{q-1}[u] + w(u, v)) \right\}.$$

The first term is essential: it preserves solutions that use fewer than q edges. The answer is $A_k[t]$.

5.2.2 Algorithm

```
Algorithm ShortestPathAtMostK(G, s, t, k):
    for each vertex x in V:
        A[0][x] = +infinity
    A[0][s] = 0
    for q = 1 to k:
        for each vertex x in V:
            A[q][x] = A[q-1][x]
        for each edge (u, v) in E:
            if A[q-1][u] != +infinity:
                A[q][v] = min(A[q][v], A[q-1][u] + w(u, v))
    if A[k][t] == +infinity:
        return NoPath
    return A[k][t]
```

Listing 5.1: Minimum-weight path using at most k edges

5.2.3 Correctness Proof

Theorem 5.1. *Algorithm 5.1 returns the minimum weight of a path from s to t that uses at most k edges, or reports that no such path exists.*

Proof. We prove by induction on q that $A_q[x]$ is the minimum weight of a path from s to x using at most q edges. For $q = 0$, only the empty path from s to itself is permitted, so the initialization is correct.

Assume the claim holds for $q - 1$. Any path using at most q edges either uses at most $q - 1$ edges, represented by $A_{q-1}[v]$, or consists of a path to some predecessor u using at most $q - 1$ edges followed by edge (u, v). By the inductive hypothesis, the minimum candidate for the latter case is $A_{q-1}[u] + w(u, v)$. The recurrence takes the minimum over these two cases and all incoming edges, so it computes the correct value for $A_q[v]$. By induction, $A_k[t]$ is the required optimum. An infinite value means no path with at most k edges exists. □

5.2.4 Complexity Analysis

Each of the k rounds copies $|V|$ values and scans all $|E|$ edges. The running time is $O(k(|V| + |E|))$. Storing every row requires $O(k|V|)$ space. If only the optimum value is needed, two rows suffice, reducing the auxiliary space to $O(|V|)$.

5.3 Paths with Exactly k Edges

5.3.1 State Definition and Recurrence

Define $B_q[x]$ as the minimum weight of a path from s to x using exactly q edges. Initialize

$$B_0[x] = \begin{cases} 0, & x = s, \\ +\infty, & x \neq s. \end{cases}$$

For $q \geq 1$, every path to v with exactly q edges must end with an edge (u, v) whose preceding subpath uses exactly $q - 1$ edges. Therefore,

$$B_q[v] = \min_{(u,v)\in E} \left(B_{q-1}[u] + w(u, v)\right),$$

where the minimum over an empty set is $+\infty$. Unlike the at-most-k recurrence, this recurrence does not carry forward $B_{q-1}[v]$, since that would permit a solution using fewer than q edges.

5.3.2 Algorithm

```
Algorithm ShortestPathExactlyK(G, s, t, k):
    for each vertex x in V:
        B[0][x] = +infinity
    B[0][s] = 0
    for q = 1 to k:
        for each vertex x in V:
            B[q][x] = +infinity
        for each edge (u, v) in E:
            if B[q-1][u] != +infinity:
                B[q][v] = min(B[q][v], B[q-1][u] + w(u, v))
    if B[k][t] == +infinity:
        return NoPath
    return B[k][t]
```

Listing 5.2: Minimum-weight path using exactly k edges

5.3.3 Correctness Proof

Theorem 5.2. *Algorithm 5.2 returns the minimum weight of a path from s to t using exactly k edges, or reports that no such path exists.*

Proof. We prove by induction on q that $B_q[x]$ is the minimum weight of a path from s to x using exactly q edges. The base case $q = 0$ is correct because the only zero-edge path begins and ends at s.

Assume the claim holds for $q - 1$. Every path to v using exactly q edges consists of a path to some vertex u using exactly $q - 1$ edges, followed by an edge (u, v). The algorithm considers every edge and combines its weight with $B_{q-1}[u]$. Taking the minimum over all such edges therefore gives the optimal value for exactly q edges. If all candidates are infinite, no such path exists. The inductive claim follows, and setting $q = k$ proves the theorem. □

5.3.4 Complexity Analysis

Each round initializes $|V|$ states and examines $|E|$ edges. The running time is $O(k(|V|+|E|))$, and storing all rows requires $O(k|V|)$ space. As with the at-most-k variant, two consecutive rows suffice when only the optimal value is required.

5.4 Comparison of the Two Dynamic Programs

Table 5.1 summarizes the main distinction.

Table 5.1: Comparison of edge-count-constrained shortest-path formulations

Property	At most k edges	Exactly k edges
State	$A_q[v]$	$B_q[v]$
Carry previous state?	Yes, $A_{q-1}[v]$	No
Edge transition	From $A_{q-1}[u]$	From $B_{q-1}[u]$
Final answer	$A_k[t]$	$B_k[t]$
Time complexity	$O(k(\|V\| + \|E\|))$	$O(k(\|V\| + \|E\|))$
Full-table space	$O(k\|V\|)$	$O(k\|V\|)$

The at-most formulation retains the best solution found using fewer edges, while the exactly formulation restricts each state to paths whose edge count matches its index. This

difference in state semantics is the central correctness condition for the two algorithms.

Chapter 6

Comparative Discussion and Limitations

6.1 Summary of Theoretical Complexity

Table 6.1: Summary of the algorithms analyzed in this thesis

Problem	Running time	Auxiliary space
Alphabet reconstruction	$O(nm + \|\Sigma\| + \|E\|)$	$O(\|\Sigma\| + \|E\|)$
DAG shortest paths	$O(\|V\| + \|E\|)$	$O(\|V\|)$
DAG longest paths	$O(\|V\| + \|E\|)$	$O(\|V\|)$
At most k edges	$O(k(\|V\| + \|E\|))$	$O(k\|V\|)$
Exactly k edges	$O(k(\|V\| + \|E\|))$	$O(k\|V\|)$

For alphabet reconstruction, n is the number of words, m is the maximum word length, and $|\Sigma|$ is the alphabet size. For the path problems, $|V|$ and $|E|$ are the graph size parameters. The dynamic programming space bounds refer to storing all states; rolling arrays can reduce auxiliary space when only the final optimum value is required.

6.2 Assumptions and Implementation Considerations

Several conditions should be made explicit in an implementation:

- **Alphabet reconstruction:** a cycle means the inferred constraints are inconsistent. A valid topological ordering may not be unique. Prefix cases must be handled if word lengths differ.

- **DAG path algorithms:** the graph must be acyclic. Unreachable vertices must not be relaxed from infinite values. Negative edge weights are allowed.
- **Edge-count dynamic programming:** the value of k controls the number of rounds. The recurrences apply to directed graphs generally, not only DAGs. An unreachable destination is reported when the final state remains infinite.
- **Numerical representation:** a concrete implementation should choose a numeric type and infinity sentinel that cannot overflow during valid additions. This detail depends on the programming language and weight bounds.

6.3 Theoretical Evaluation and Empirical Limitations

The complexity bounds are derived from the operations specified in the algorithms. This thesis does not include benchmark measurements, generated graph datasets, or a controlled implementation comparison. Therefore, it makes no claim about observed execution times or practical performance differences. Such claims would require a reproducible experimental protocol, documented input generation, implementation details, repeated measurements, and presentation of the resulting data.

A useful future experiment could compare measured runtimes across graphs with varying vertex counts, edge densities, weight distributions, and values of k. The experiments should distinguish graph construction time from algorithm execution time and should report hardware, software versions, and measurement methodology. Until such an evaluation is conducted, the conclusions remain theoretical.

6.4 Limitations

The analysis focuses on correctness and asymptotic complexity rather than novel algorithm development. The alphabet reconstruction method returns an ordering consistent with inferred constraints but cannot promise a unique order unless the input constraints determine one. The path algorithms are described using standard adjacency-list operations and constant-time arithmetic; implementation-level performance can vary with data structures, numeric representation, and input characteristics. Finally, no external literature review has been completed for this draft, so it should not be treated as a fully referenced scholarly submission until verified sources and institutional requirements are added.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This thesis has presented four graph-related algorithmic tasks and analyzed their design, correctness, and computational requirements. Alphabet reconstruction converts lexical comparisons into precedence edges and uses topological sorting to produce a consistent symbol order. The method detects inconsistent constraints through cycles, while uniqueness requires an additional condition on the resulting partial order.

For weighted DAGs, topological processing enables shortest- and longest-path computation in $O(|V| + |E|)$ time. These algorithms support negative edge weights because acyclicity guarantees that predecessor values can be finalized in topological order. For paths constrained by edge count, dynamic programming achieves $O(k(|V|+|E|))$ time. The at-most-k recurrence preserves a previous solution using fewer edges, whereas the exactly-k recurrence only extends states from the preceding edge count.

The central conclusion is that correct state definitions and graph structure determine which algorithmic strategy is appropriate. Topological ordering is effective when acyclicity supplies a valid processing order, while edge-count dynamic programming provides a bounded sequence of subproblems even in graphs that contain cycles.

7.2 Future Work

Several extensions are appropriate for further study:

1. implement the algorithms in a common programming language and validate them against independently constructed test cases;

2. conduct reproducible experiments to compare observed runtime and memory use with the theoretical bounds;

3. extend alphabet reconstruction to report whether the inferred order is unique and to provide informative diagnostics for inconsistent input;

4. compare full-table and rolling-array implementations of the edge-count dynamic programs; and

5. investigate additional constraints, such as path reconstruction, multiple sources and destinations, or application-specific restrictions on edge weights.

These extensions would supplement the current theoretical analysis with empirical evidence and a broader evaluation of the algorithms' practical use.

References

No external references are listed in this draft because verified bibliographic details were not supplied. Before submission, this section should be populated with sources actually consulted, such as an appropriate algorithms textbook and primary sources for the graph algorithms discussed. All citations and bibliographic entries should be checked against the institution's required citation style. Do not submit this placeholder as a completed references section.

Appendix A

Suggested Verification Checklist

The following checklist can be used when implementing and validating the algorithms. It is a proposed plan, not a report of experiments already performed.

1. For alphabet reconstruction, test a valid uniquely ordered instance, a valid instance with multiple possible orders, a cycle of contradictory constraints, and an invalid prefix ordering when words have different lengths.

2. For DAG shortest paths, include positive, zero, and negative edge weights, an unreachable vertex, and a source with no outgoing edges.

3. For DAG longest paths, include a graph where the optimal path is not the path with the most edges, and include negative weights and unreachable vertices.

4. For at-most-k paths, test $k = 0$, no feasible path, a direct edge, and a case where the best path uses fewer than k edges.

5. For exactly-k paths, test $k = 0$, no feasible path of exactly k edges, and a case where a feasible path with fewer than k edges must not be accepted.

6. Compare each computed result with exhaustive enumeration on small graphs, where exhaustive enumeration is feasible.